



TRABALHO PRÁTICO I

PROCESSAMENTO DE LINGUAGEM NATURAL

pg59861 Bárbara Grandinho
pg59892 Laura Ferreira

16/04/2026

GLOSSÁRIO
MINISTÉRIO DE SAÚDE

GLOSSÁRIO
NEOLOGISMOS

WIPOPEARL
COVID-19 GLOSSARY

MEDICINA
GERAL

1

2

3

4

**DOCUMENTOS
UTILIZADOS**

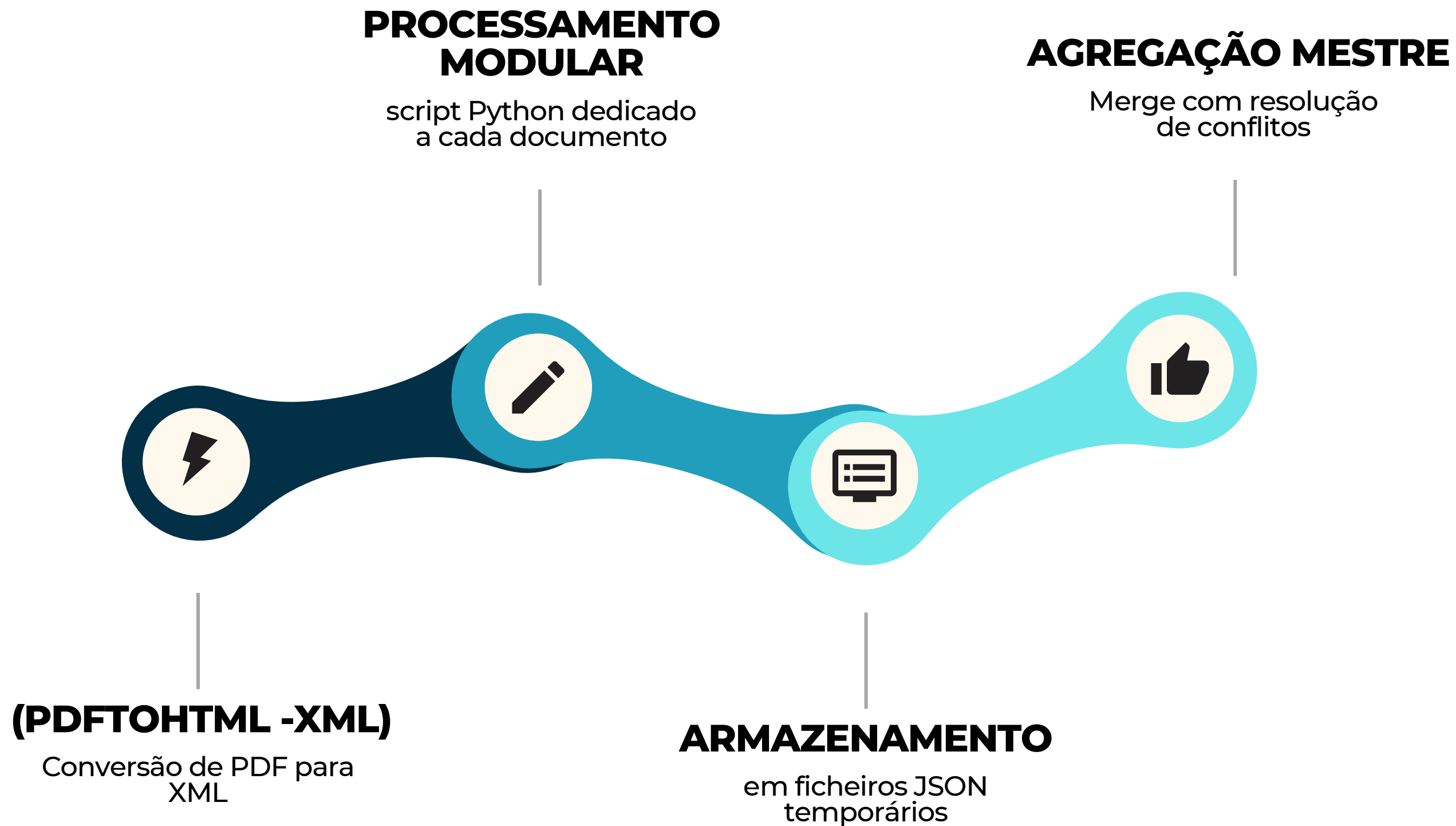
OBJETIVO

Extração automatizada de conceitos médicos, descrições e traduções a partir de documentos não estruturados (PDFs).

**MEDICINA.JSON + WIPOPEARL.JSON + MINISTERIO_SAUDE.JSON + NEOLOGISMOS.JSON + SIGLAS_GERAL.JSON +
ABREVIATURAS_GERAIS.JSON**

Construção de um dicionário digital unificado e pesquisável em formato JSON.

ARQUITETURA GERAL DO PROJETO



GLOSSÁRIO

MEDICINA

1

IMPORTS E LEITURA DO FICHEIRO

```
import re
import json

# 1. Abrir e ler o ficheiro XML
with open(r'C:\Users\barba\OneDrive - Universidade do Minho\Universidade\4ºano\2º Semestre\PLN\plneb-2526') as f:
    texto = f.read()
```

GLOSSÁRIO

MEDICINA

2

Limpeza de dados

```
# =====  
# FASE 1: LIMPEZA DE DADOS E MARCAÇÕES  
# =====  
  
# Remover lixo de formatação do XML (pages, fontspecs, etc.)  
texto = re.sub(r'</?page.*?>', '', texto)  
texto = re.sub(r'</?fontspec.*?>', '', texto)  
  
# =====  
# UNIR CONCEITOS PARTIDOS EM VÁRIAS LINHAS  
# Se um <b> fecha e a linha seguinte abre um <b> que não começa por um número,  
# (Corremos duas vezes para garantir que cola conceitos partidos em 3 linhas)  
# =====  
texto = re.sub(r'</b>\s*</text>\s*<text[^>]*>\s*<b>\s*(?!\\d+\\s+)', ' ', texto)  
texto = re.sub(r'</b>\s*</text>\s*<text[^>]*>\s*<b>\s*(?!\\d+\\s+)', ' ', texto)
```

GLOSSÁRIO

MEDICINA

3

Marcar Conceitos

```
# 1. MARCAR O CONCEITO PRINCIPAL (2 CASOS DE FORMATAÇÃO NO XML):
# CASO A: O número está DENTRO da tag <b>
texto = re.sub(r'<text[^>]*>\s*<b>\s*(\d+)\s+(.*?)\s*</b>\s*</text>', r'@@CONCEITO:\1|\2@@', texto)

# CASO B: O número ficou numa tag <text> separada imediatamente do <b>
texto = re.sub(r'<text[^>]*>\s*(\d+)\s*</text>\s*<text[^>]*>\s*<b>\s*(.*?)\s*</b>\s*</text>', r'@@CONCEITO:\1|\2@@', texto)

# 1.1 LIMPAR A CLASSE GRAMATICAL DO CONCEITO:
# Capturamos o ID e o Nome no Grupo 1, e ignoramos o espaço e a classe no final
# Executamos duas vezes para o caso de haverem duas classes seguidas
texto = re.sub(r'(\d+|\.*)\s+(?:m|f|a|m\s+pl|f\s+pl|pl|s|sg|abrev\.)@@', r'\1@@', texto)
texto = re.sub(r'(\d+|\.*)\s+(?:m|f|a|m\s+pl|f\s+pl|pl|s|sg|abrev\.)@@', r'\1@@', texto)

# 1.2 MARCAR CONCEITOS EXTRA/SECUNDÁRIOS COMO LIXO:
# Qualquer tag <b> que não comece por um número é transformada numa "parede"
texto = re.sub(r'<text[^>]*>\s*<b>\s*(?! \d+ \s+)(.*?)\s*</b>\s*</text>', r'@@LIXO:', texto)
```

GLOSSÁRIO

MEDICINA

4

Marcar categorias, traduções , sinónimos e variantes

```
# 2. MARCAR A CATEGORIA:
# A maioria está na font="21".
texto = re.sub(r'<text[^>]*font="21"[^>]*>\s*<i>\s*(.*?)\s*</i>\s*</text>', r'@@CATEGORIA:\1@@', texto)
# Regra font=22: Exige uma Maiúscula seguida obrigatoriamente de uma minúscula, porque as traduções estão na mesma font, mas começam
texto = re.sub(r'<text[^>]*font="22"[^>]*>\s*<i>\s*([A-ZÁÉÍÓÚŃÇ][a-záéíóúñç]+.*?)\s*</i>\s*</text>', r'@@CATEGORIA:\1@@', texto)

# 3. MARCAR OS IDIOMAS (traduções):
texto = re.sub(r'<text[^>]*>\s*es\s*</text>', r'@@ES:', texto)
texto = re.sub(r'<text[^>]*>\s*en\s*</text>', r'@@EN:', texto)
texto = re.sub(r'<text[^>]*>\s*pt\s*</text>', r'@@PT:', texto)
texto = re.sub(r'<text[^>]*>\s*la\s*</text>', r'@@LA:', texto)

# 4. MARCAR SINÓNIMOS, VARIANTES E NOTAS:
texto = re.sub(r'SIN\.-', '@@SIN:', texto)
texto = re.sub(r'VAR\.-', '@@VAR:', texto)
texto = re.sub(r'Nota\.-', '@@NOTA:', texto)
```

GLOSSÁRIO MEDICINA

5

Limpeza final do XML

```
# 5. LIMPEZA FINAL DO XML:
# Substitui qualquer tag XML restante por um espaço e remove espaços duplicados
texto = re.sub(r'<[^>]+>', ' ', texto)
texto = re.sub(r'\s+', ' ', texto)

# 6. LIMPAR CABEÇALHOS DE PÁGINA:
texto = re.sub(r'V\s*ocabulario\s*\d*', '@@LIXO:', texto, flags=re.IGNORECASE)

# 7. CORTAR AS EXTREMIDADES DO DOCUMENTO (Introdução e Índices Finais):

start_match = re.search(r'@@CONCEITO:1\|á\s*@@', texto)
if start_match:
    texto = texto[start_match.start():]

end_match = re.search(r'Í\s*ndice\s+de\s+denominacións', texto, flags=re.IGNORECASE)
if end_match:
    texto = texto[:end_match.start()]
```


GLOSSÁRIO MEDICINA

6

Extração de Informação

```
# =====  
# FASE 2: EXTRAÇÃO DA INFORMAÇÃO  
# =====  
  
dicionario_medicina = {}  
  
# O split divide o texto gigante num bloco para cada conceito.  
# Ignoramos o índice 0 porque corresponde ao cabeçalho antes do primeiro conceito.  
blocos = texto.split("@@CONCEITO:")[1:]  
  
for bloco in blocos:  
    # 1. Extrair o ID e o Conceito  
    conceito_match = re.search(r'^(\d+)\|(.*)@', bloco)  
    if not conceito_match:  
        continue  
  
    id_conceito = conceito_match.group(1)  
    conceito = conceito_match.group(2).strip()  
  
    if not conceito:  
        continue
```

GLOSSÁRIO MEDICINA

7

Extração de Categorias, Sinónimos e Variantes

```
# 2. Extrair Categoria
categorias_cruas = re.findall(r'@@CATEGORIA:(.*?)@@', bloco)
categorias = []

for c in categorias_cruas:
    c = c.strip()
    if c: # Se não for uma categoria vazia
        # Cortar quando uma minúscula choca com uma maiúscula
        c_separado = re.sub(r'([a-záéíóúñ])\s+([A-ZÁÉÍÓÚÑ])', r'\1|\2', c)
        # Agora divide pelo símbolo | que acabámos de criar
        partes = c_separado.split('|')
        categorias.extend(partes)

if not categorias:
    categorias = "Categoria não identificada"

# 3. Extrair Sinónimos (SIN)
sin_match = re.search(r'@@SIN:(.*?)(?=@@|$)', bloco)
sinonimos = [s.strip() for s in sin_match.group(1).split(';')] if sin_match else []
if not sinonimos:
    sinonimos = "Sinónimos não identificados"

# 4. Extrair Variantes (VAR)
var_match = re.search(r'@@VAR:(.*?)(?=@@|$)', bloco)
variantes = [v.strip() for v in var_match.group(1).split(';')] if var_match else []
if not variantes:
    variantes = "Variantes não identificadas"
```


GLOSSÁRIO

MEDICINA

8

Extração de Descrições e Traduções

```
# 5. Extrair Nota/Descrição
nota_match = re.search(r'@@NOTA:(.*?)(?=@@|$)', bloco)
nova_descricao = nota_match.group(1).strip() if nota_match else "Descrição não identificada"

# 6. Extrair Traduções
traducoes = {}
idiomas = [("ES", "@@ES:"), ("EN", "@@EN:"), ("PT", "@@PT:"), ("LA", "@@LA:")]
for lang, tag in idiomas:
    # A captura para sempre que encontra o próximo @@ ou o fim da string
    lang_match = re.search(rf'{tag}(.*?)(?=@@|$)', bloco)
    if lang_match:
        # Limpa o texto e normaliza a pontuação
        traducao_limpa = lang_match.group(1).strip().replace(" ; ", "; ")
        traducoes[lang] = traducao_limpa
if not traducoes:
    traducoes = "Traduções não identificadas"
```

GLOSSÁRIO MEDICINA

9

```
# =====  
# ESTRUTURA DOS DADOS EM JSON C/ TRATAMENTO DE REPETIÇÕES E HOMONÍMIA  
# =====  
  
# Primeiro, procurar todas as chaves existentes no dicionário que derivem deste conceito  
chaves_existentes = []  
if conceito in dicionario_medicina:  
    chaves_existentes.append(conceito)  
for k in dicionario_medicina.keys():  
    if re.match(rf'^\\(\\d+\\)\\s+{re.escape(conceito)}$', k):  
        chaves_existentes.append(k)  
  
if not chaves_existentes:  
    # É a primeira vez que vemos o conceito, guardamos normalmente  
    dicionario_medicina[conceito] = {  
        "categoria": categorias,  
        "sinonimos": sinonimos,  
        "variantes": variantes,  
        "descricao": nova_descricao,  
        "traducoes": traducoes  
    }  
else:  
    # O conceito já existe! Vamos verificar se faz match perfeito com algum dos existentes  
    chave_correspondente = None  
    for chave in chaves_existentes:  
        dados_existentes = dicionario_medicina[chave]  
        if (dados_existentes["categoria"] == categorias and  
            dados_existentes["sinonimos"] == sinonimos and  
            dados_existentes["variantes"] == variantes and  
            dados_existentes["traducoes"] == traducoes):  
            chave_correspondente = chave  
            break
```

Extração de Descrições e Traduções

```
if chave_correspondente:  
    # Se os parâmetros são iguais, e a nova descrição for válida e diferente, fundimos as descrições  
    desc_atual = dicionario_medicina[chave_correspondente]["descricao"]  
  
    if nova_descricao != "Descrição não identificada" and nova_descricao not in desc_atual:  
  
        if desc_atual == "Descrição não identificada":  
            dicionario_medicina[chave_correspondente]["descricao"] = nova_descricao  
  
        elif not desc_atual.startswith("(1)":  
            dicionario_medicina[chave_correspondente]["descricao"] = f"(1) {desc_atual} (2) {nova_descricao}"  
  
        else:  
            qtd_existentes = len(re.findall(r'\\(\\d+\\)', desc_atual))  
            dicionario_medicina[chave_correspondente]["descricao"] = f"{desc_atual} ({qtd_existentes + 1}) {nova_descricao}"  
  
else:  
    # Se os parâmetros forem diferentes (Homonímia) -> Criar "(1) ..." e "(2) ..."  
    if conceito in chaves_existentes:  
        dicionario_medicina[f"(1) {conceito}"] = dicionario_medicina.pop(conceito)  
        chaves_existentes.remove(conceito)  
        chaves_existentes.append(f"(1) {conceito}")  
  
    proximo_numero = len(chaves_existentes) + 1  
    nova_chave = f"({proximo_numero}) {conceito}"  
  
    dicionario_medicina[nova_chave] = {  
        "categoria": categorias,  
        "sinonimos": sinonimos,  
        "variantes": variantes,  
        "descricao": nova_descricao,  
        "traducoes": traducoes  
    }
```

GLOSSÁRIO

MEDICINA

10

Exportação do JSON

```
# =====  
# FASE 3: EXPORTAR PARA JSON  
# =====  
  
with open("jsons_temporarios/medicina.json", "w", encoding="utf-8") as f_out:  
    json.dump(dicionario_medicina, f_out, ensure_ascii=False, indent=4)  
  
print(f"Processamento limpo concluído! Foram extraídos {len(dicionario_medicina)} conceitos.")
```

GLOSSÁRIO

MEDICINA

11

```
{
  "á": {
    "categoria": [
      "Anatomía"
    ],
    "sinonimos": "Sinónimos não identificados",
    "variantes": "Variantes não identificadas",
    "descricao": "Descrição não identificada",
    "traducoes": {
      "ES": "ala",
      "EN": "wing",
      "PT": "asa",
      "LA": "ala"
    }
  },
  "a termo": {
    "categoria": [
      "Fisioloxía"
    ],
    "sinonimos": "Sinónimos não identificados",
    "variantes": "Variantes não identificadas",
    "descricao": "Descrição não identificada",
    "traducoes": {
      "ES": "a término",
      "EN": "full term",
      "PT": "a termo"
    }
  }
},
```

medicina.json

```
"zume gástrico": {
  "categoria": [
    "Fisioloxía"
  ],
  "sinonimos": "Sinónimos não identificados",
  "variantes": "Variantes não identificadas",
  "descricao": "Descrição não identificada",
  "traducoes": {
    "ES": "jugo gástrico",
    "EN": "gastric juice",
    "PT": "suco gástrico"
  }
},
"zume pancreático": {
  "categoria": [
    "Fisioloxía"
  ],
  "sinonimos": "Sinónimos não identificados",
  "variantes": "Variantes não identificadas",
  "descricao": "Descrição não identificada",
  "traducoes": {
    "ES": "jugo pancreático",
    "EN": "pancreatic juice",
    "PT": "suco pancreático"
  }
}
}
```


GLOSSÁRIO

MINISTÉRIO SAÚDE

1

IMPORTS E LEITURA DO FICHEIRO

```
import re
import json

def extrair_ministerio_saude(caminho_xml):

    print(f"[{caminho_xml}] A iniciar extração ...")

    with open(caminho_xml, 'r', encoding='utf-8') as f:
        texto = f.read()
```


GLOSSÁRIO

MINISTÉRIO SAÚDE

2

Pré-Processamento e limpeza dos dados

```
# 1. Pré-Processamento: limpeza tipográfica no xml
# Remove tags estruturais irrelevantes (páginas, metadados do conversor e imagens)
texto = re.sub(r"</?page.*?>", r"", texto)
texto = re.sub(r"</?pdf2xml.*?>", r"", texto)
texto = re.sub(r"</?image.*?>", r"", texto)

# Remoção de estruturas visuais baseada no tamanho da fonte (font size).
# Elimina números de página no rodapé (15) e siglas de navegação lateral (22, 23, 25),
# impedindo a sua concatenação indevida com o corpo do texto.
padrao_fontes_lixo = r'<text[^>]*font="(22|15|25|23)"[^>]*>.*?</text>\n?'
texto = re.sub(padrao_fontes_lixo, r"", texto)

# 2. Normalização textual
# Resolução de ligaturas tipográficas geradas pelo conversor PDF
texto = re.sub(r'fi', 'fi', texto)
texto = re.sub(r'fl', 'fl', texto)
texto = re.sub(r'fi\s+([a-zA-ÿ])', r'fi\1', texto)

# Padronização de espaçamentos (remoção de quebras de linha estritas)
texto = re.sub(r'\n', ' ', texto)
texto = re.sub(r'\s+', ' ', texto)
```

GLOSSÁRIO

MINISTÉRIO SAÚDE

3

Segmentação de Entidades

```
# 3. Ancoragem e segmentação de entidades

# Correção de quebras artificiais em termos a negrito
texto = re.sub(r'</b></text>\s*<text[^>]*>\s*<b>', ' ', texto)
# Inserção de marcadores estruturais baseados no padrão visual do documento
# (Fonte 21 a negrito, ou negrito imediatamente seguido de "Categoria" ou "Ver")
texto = re.sub(r'<text[^>]*font="21"[^>]*><b>(.*?)</b></text>', r'###TERMO###\1', texto)
texto = re.sub(r'<b>(.*?)</b></text>\s*<text[^>]*>\s*(?:Categoria|Ver)', r'###TERMO###\1 <text>Categoria', texto)

# Eliminar do documento as restantes tags XML, convertendo-o em texto simples
texto = re.sub(r'<text[^>]*>|</text>|<i>|</i>|<b>|</b>|<fontspec[^>]*>', ' ', texto)
texto = re.sub(r'\s+', ' ', texto).strip()

# Reconstrução de vocábulos fragmentados por hifenização de quebra de linha
texto = re.sub(r'([a-zA-ZÀ-ÿ])-( [a-zA-ZÀ-ÿ])', r'\1\2', texto)

# Isolamento do corpo de dados: Descartar o índice e introdução
partes_inicio = re.split(r'###TERMO###Abordagem médica', texto, maxsplit=1)
if len(partes_inicio) > 1:
    texto = "###TERMO###Abordagem médica" + partes_inicio[1]

# Segmentação do texto contínuo numa lista de instâncias independentes
blocos = re.split(r'###TERMO###', texto)

dicionario_final = {}
```

GLOSSÁRIO

MINISTÉRIO SAÚDE

Extração iterativa de atributos

4

```
# 4. Extração Iterativa de atributos
```

```
for bloco in blocos[1:]:
```

```
    bloco = bloco.strip()
    if len(bloco) < 3: continue
```

```
    # 4.1 Tratamento de Referências Cruzadas (Cross-references)
```

```
    match_ver = re.search(r'\s+Ver\s+', bloco, re.IGNORECASE)
```

```
    if match_ver:
```

```
        partes = re.split(re.escape(match_ver.group(0)), bloco, maxsplit=1)
        termo = re.sub(r'Categoria\s*:\?', '', partes[0]).strip()
        if len(termo) > 100: continue # Filtro de ruído
```

```
        dicionario_final[termo] = {
            "categoria": ["Referência Cruzada"],
            "descricao": "Ver " + partes[1].strip()
        }
```

```
    continue
```

```
# 4.2 Extração de Conceitos Regulares
```

```
pos_cat = bloco.lower().find("categoria")
```

```
if pos_cat != -1:
    termo = bloco[:pos_cat].strip()
```

```
    # Filtro dimensional para mitigar a captura de índices residuais
    if len(termo) > 100:
        continue
```

```
    resto = bloco[pos_cat:].strip()
```

```
    # Limpeza do descritor semântico "Categoria:"
    resto = re.sub(r'^categoria\s*:\?', '', resto, flags=re.IGNORECASE)
```

```
    # Correções ortográficas inerentes a erros de OCR/Tipografia do PDF
    resto = resto.replace("Demografi a", "Demografia")
    resto = resto.replace("Economia da Saúde", "Economia de Saúde")
    resto = resto.replace("Ciências Sociais e Saúde", "Ciências Sociais em Saúde")
```

```
    # Identificação e extração da área temática
    cats_encontradas = []
```

```
    for cat in categorias_oficiais:
```

```
        if resto.lower().startswith(cat.lower()):
            cats_encontradas.append(cat)
            resto = resto[len(cat):].strip()
            break
```

```
    # O que sobra é a descrição
    descricao = resto.strip()
```

GLOSSÁRIO

MINISTÉRIO SAÚDE

5

Pós-Processamento e recuperação de oclusões

```
# 5. Pós-Processamento e recuperação de oclusões

# Algoritmo desenhado para recuperar títulos e categorias que, devido à
# ausência de formatação a negrito no PDF, ficaram embutidos na definição do termo anterior.
termos_a_adicionar = {}

for termo, info in dicionario_final.items():
    descricao = info["descricao"]

    # Expressão regular para detetar: [Início de Frase] + "Categoria:" + [Continuação]
    match = re.search(r'([A-Z][A-Za-zÀ-ÿ\s]+)\s+Categoria\s*:\s*([A-Za-zÀ-ÿ\s,]+\s+)([A-Z].*)', descricao)
    if match:
        novo_termo = match.group(1).strip()
        # Remoção de lixo de cabeçalho anexado ao novo termo
        novo_termo = re.sub(r'^[A-Z]{3,}\s+', '', novo_termo).strip()
        nova_cat = match.group(2).strip()
        nova_def = match.group(3).strip()

        # Truncamento da definição original no ponto exato da oclusão
        texto_antes = descricao[:match.start()].strip()
        texto_antes = re.sub(r'\s+[A-Z]{3,}$', '', texto_antes).strip()
        dicionario_final[termo]["descricao"] = texto_antes

        # Preparação da nova instância resgatada
        termos_a_adicionar[novo_termo] = {
            "categoria": [nova_cat] if nova_cat in categorias_oficiais else [],
            "descricao": nova_def
        }

# Adiciona os termos ao dicionário principal
dicionario_final.update(termos_a_adicionar)
```

GLOSSÁRIO

MINISTÉRIO SAÚDE

6

Exportação do JSON

```
# 7. Exportação do json
f_out = open("jsons_temporarios/ministerio_saude_temp.json", "w", encoding="utf-8")
json.dump(dicionario_final, f_out, indent=4, ensure_ascii=False)
f_out.close()

print(f"Concluído! Extraídos {len(dicionario_final)} conceitos.")
return dicionario_final

extrair_ministerio_saude("dados/glossario_ministerio_saude.xml")
```


GLOSSÁRIO

MINISTÉRIO SAÚDE

7

ministerio_saude.json

```
{
  "Antibiótico": {
    "categoria": [
      "Medicamentos, vacinas e insumos"
    ],
    "descricao": "Antimicrobianos produzidos por microrganismos."
  },
  "Antibioticoterapia": {
    "categoria": [
      "Medicamentos, vacinas e insumos"
    ],
    "descricao": "É o tratamento de pacientes com sinais e sin-"
  },
  "Anticoncepção": {
    "categoria": [
      "Referência Cruzada"
    ],
    "descricao": "Ver Métodos contraceptivos."
  },
}
```

GLOSSÁRIO

WIPOPEARL

1

Exportação do JSON

```
import re
import json

# 1. Abrir e ler o ficheiro XML completo
caminho_xml = r'C:\Users\barba\OneDrive - Universidade do Minho\Universidade\4ºano\2º Semestre\PLN\plneb-2526'

with open(caminho_xml, 'r', encoding='utf-8') as f:
    texto = f.read()
```

GLOSSÁRIO

WIPOPEARL

2

Delimitar o documento

```
# =====  
# CORTAR AS EXTREMIDADES DO DOCUMENTO  
# =====  
  
# Cortar tudo o que está para trás do título do glossário  
start_match = re.search(r'<b>Multilingual Glossary </b>', texto)  
if start_match:  
    texto = texto[start_match.end():]  
  
# Cortar o rodapé final comum da última página para excluir os metadados  
end_match = re.search(r'World Intellectual Property Organization', texto)  
if end_match:  
    texto = texto[:end_match.start()]
```

GLOSSÁRIO

WIPOPEARL

3

Limpeza de dados e marcações táticas

```
# =====
# FASE 1: LIMPEZA DE DADOS E MARCAÇÕES TÁTICAS
# =====

# Remover tags estruturais desnecessárias que sujam o texto
texto = re.sub(r'</?page.*?>', '', texto)
texto = re.sub(r'</?fontspec.*?>', '', texto)

# 1. MARCAR O CONCEITO: Sabemos pelo PDF que os conceitos base estão sempre na font 8 a negrito
texto = re.sub(r'<text[^>]*font="8"[^>]*>\s*<b>\s*(.*?)\s*</b>\s*</text>', r'@@CONCEITO:\1@@', texto)

# 2. MARCAR A CATEGORIA: As categorias estão sempre na font 11.
texto = re.sub(r'<text[^>]*font="11"[^>]*>\s*(.*?)\s*</text>', r'@@CATEGORIA:\1@@', texto)

# 3. MARCAR DESCRIÇÃO: O texto da descrição usa a font 6.
# Como as descrições ocupam várias linhas, marcamos cada linha individualmente. Na Fase 2 vamos colá-las.
texto = re.sub(r'<text[^>]*font="6"[^>]*>\s*(.*?)\s*</text>', r'@@DESC:\1@@', texto)

# 4. MARCAR IDIOMAS: As siglas das línguas estão na font 7 a negrito.
# Capturamos apenas as siglas oficiais para evitar falsos positivos
texto = re.sub(r'<text[^>]*font="7"[^>]*>\s*<b>\s*(AR|DE|ES|FR|JA|KO|PT|RU|ZH)\s*</b>\s*</text>', r'@@LANG:\1@@', texto)

# 5. LIMPEZA FINAL DO XML: Remoção de tags XML residuais e normalização de espaços em branco consecutivos.
texto = re.sub(r'<[^>]+>', ' ', texto)
texto = re.sub(r'\s+', ' ', texto)
```

GLOSSÁRIO

WIPOPEARL

Extração e Processamento de Dados

4

```
# =====
# FASE 2: EXTRAÇÃO E ESTRUTURAÇÃO
# =====

dicionario_wipo = {}

# Ignorar o índice [0] porque é o espaço vazio antes do primeiro conceito
blocos = texto.split("@@CONCEITO:")[1:]

for bloco in blocos:
    # 1. Extrair o Conceito Base
    # O conceito está logo no início do bloco, até ao primeiro "@@"
    conceito_match = re.search(r'^(.*)@@', bloco)
    if not conceito_match:
        continue

    conceito = conceito_match.group(1).strip()
    # Guardamos o resto do bloco (tudo o que vem depois do conceito) para pesquisar
    bloco_restante = bloco[conceito_match.end():]

    # 2. Extrair Categorias
    cat_match = re.search(r'@@CATEGORIA:(.*)@@', bloco_restante)
    if cat_match:
        # As categorias neste documento vêm separadas por vírgula
        categorias = [c.strip() for c in cat_match.group(1).split(',')]
    else:
        categorias = ["Categoria não identificada"]
```

```
# 3. Extrair Descrição
# Encontramos todas as linhas marcadas com @@DESC: e unimos com um espaço
desc_matches = re.findall(r'@@DESC:(.*)@@', bloco_restante)
nova_descricao = " ".join([d.strip() for d in desc_matches]).strip()
if not nova_descricao:
    nova_descricao = "Descrição não identificada"

# 4. Extrair Sinónimos Principais
# Para não extrair os (syn.) das traduções, isolamos apenas a string que e
desc_pos = bloco_restante.find("@@DESC:")
cat_pos = bloco_restante.find("@@CATEGORIA:")

limites = [pos for pos in [desc_pos, cat_pos] if pos != -1]
limite_header = min(limites) if limites else len(bloco_restante)

# "header" contém apenas o espaço onde mora o sinónimo inglês
header = bloco_restante[:limite_header]

# Procuramos literalmente pela palavra (syn.) apenas no cabeçalho
sin_match = re.search(r'\(syn\.\)\s*(.*)', header)
if sin_match:
    sin_raw = sin_match.group(1).strip()
    sinonimos = [s.strip() for s in sin_raw.split(',') if s.strip()]
else:
    sinonimos = "Sinónimos não identificados"
```


GLOSSÁRIO

WIPOPEARL

5

Extração e Processamento de Dados

```
# 5. Extrair Traduções
traducoes = {}
# O re.split divide a string sempre que encontra um idioma.
# Exemplo de output: ['texto solto', 'ES', 'tradução espanhola', 'PT', 'tradução PT']
partes_lang = re.split(r'@@LANG:([A-Z]{2})@@', bloco_restante)

# Saltamos o índice 0 e iteramos de 2 em 2 (Código do idioma -> Texto da tradução)
for i in range(1, len(partes_lang), 2):
    lang_code = partes_lang[i]
    lang_text = partes_lang[i+1].strip()

    # O lang_text mantém-se 100% fiel ao original (incluindo as vírgulas e os "(syn.)"
    if lang_text:
        traducoes[lang_code] = lang_text

if not traducoes:
    traducoes = "Traduções não identificadas"
```

GLOSSÁRIO

WIPOPEARL

Lógica de Fusão e Homonímia

6

```
chaves_existentes = []
if conceito in dicionario_wipo:
    chaves_existentes.append(conceito)
for k in dicionario_wipo.keys():
    if re.match(rf'^\\(\\d+\\)\\s+{re.escape(conceito)}$', k):
        chaves_existentes.append(k)

if not chaves_existentes:
    # É a primeira vez que vemos o conceito, guardamos normalmente
    dicionario_wipo[conceito] = {
        "categoria": categorias,
        "sinonimos": sinonimos,
        "descricao": nova_descricao,
        "traducoes": traducoes
    }
else:
    # O conceito já existe! Procurar correspondência de parâmetros
    chave_correspondente = None
    for chave in chaves_existentes:
        dados_existentes = dicionario_wipo[chave]
        if (dados_existentes["categoria"] == categorias and
            dados_existentes["sinonimos"] == sinonimos and
            dados_existentes["traducoes"] == traducoes):
            chave_correspondente = chave
            break
```

```
if chave_correspondente:
    # Fundir as notas explicativas se forem diferentes
    desc_atual = dicionario_wipo[chave_correspondente]["descricao"]
    if nova_descricao != "Descrição não identificada" and nova_descricao not in desc_atual:
        if desc_atual == "Descrição não identificada":
            dicionario_wipo[chave_correspondente]["descricao"] = nova_descricao
        elif not desc_atual.startswith("(1)"):
            dicionario_wipo[chave_correspondente]["descricao"] = f"(1) {desc_atual} (2) {nova_descricao}"
        else:
            qtd_existentes = len(re.findall(r'\\(\\d+\\)', desc_atual))
            dicionario_wipo[chave_correspondente]["descricao"] = f"{desc_atual} ({qtd_existentes + 1}) {nova_descricao}"
    else:
        # Homonímia -> Os conceitos são iguais, mas categorias/traduições diferentes.
        if conceito in chaves_existentes:
            dicionario_wipo[f"(1) {conceito}"] = dicionario_wipo.pop(conceito)
            chaves_existentes.remove(conceito)
            chaves_existentes.append(f"(1) {conceito}")

proximo_numero = len(chaves_existentes) + 1
nova_chave = f"({proximo_numero}) {conceito}"

dicionario_wipo[nova_chave] = {
    "categoria": categorias,
    "sinonimos": sinonimos,
    "descricao": nova_descricao,
    "traducoes": traducoes
}
```

GLOSSÁRIO

WIPOPEARL

7

Exportação do JSON

```
# =====  
# FASE 3: EXPORTAÇÃO JSON  
# =====  
  
with open("jsons_temporarios/wipopearl.json", "w", encoding="utf-8") as f_out:  
    json.dump(dicionario_wipo, f_out, ensure_ascii=False, indent=4)  
  
print(f"Processamento limpo concluído com sucesso! Foram extraídos {len(dicionario_wipo)} conceitos.")
```

GLOSSÁRIO

WIPOPEARL

8

wipopearl.json

```
{
  "acute respiratory distress syndrome": {
    "categoria": [
      "MEDI",
      "Pathology"
    ],
    "sinonimos": [
      "ARDS"
    ],
    "descricao": "Respiratory disease characterized by the rapid onset of widespread inflammation in the lungs.",
    "traducoes": {
      "AR": "ةمزلاتم ةقئافلا ةيسفنتلا ةداحلا",
      "DE": "akutes Atemnotsyndrom des Erwachsenen, (syn.) ARDS",
      "ES": "síndrome de dificultad respiratoria aguda, (syn.) SDRA",
      "FR": "syndrome de détresse respiratoire aiguë, (syn.) SDRA",
      "JA": "急性呼吸窮迫症候群 , (syn.) 急性呼吸促迫症候群 , ARDS",
      "KO": "급성 호흡곤란 증후군 , (syn.) ARDS",
      "PT": "síndrome do desconforto respiratório agudo, (syn.) SDRA",
      "RU": "острый респираторный дистресс-синдром, (syn.) ОРДС",
      "ZH": "急性呼吸窘迫综合征 , (syn.) ARDS"
    }
  },
}
```

GLOSSÁRIO

NEOLOGISMOS

1

Exportação do JSON

```
import re
import json

def extrair_neologismos(caminho_xml):
    print(f"[{caminho_xml}] A iniciar extração...")

    f = open(caminho_xml, 'r', encoding='utf-8')
    texto = f.read()
```


GLOSSÁRIO

NEOLOGISMOS

2

Pré-Processamento e limpeza dos dados

```
# 1. Normalização Inicial do Texto
# Substitui quebras de linha e múltiplos espaços por um único espaço
texto = re.sub(r'\n', ' ', texto)
texto = re.sub(r'\s+', ' ', texto)
texto = re.sub(r'&#34;', '"', texto) # Transforma aspas HTML em aspas normais

# 2. Remoção de Elementos Estruturais e Cabeçalhos
# Remove delimitadores de página e de imagens
texto = re.sub(r'<page[^>]*>|</page>|<image[^>]*>', ' ', texto)
# Remove cabeçalhos que contêm o mês, ano e números de página
texto = re.sub(r'<text[^>]*>\s*([A-Z][a-z]+, [A-Z][a-z]+ \d{4}|\d{2,3})\s*</text>', ' ', texto)
```

GLOSSÁRIO

NEOLOGISMOS

3

Identificação e Marcação dos Termos Principais

```
# 3. Identificação e Marcação dos Termos Principais
# Utiliza a tag <i> contendo a classe gramatical (s.m. ou s.f.) como âncora
# para isolar o neologismo e a sua respectiva classe.
texto = re.sub(r'<text[^>]*>\s*([<]+?)\s*</text>\s*<text[^>]*>\s*<i>\s*(s\.m\.|s\.f\.)\s*</i>\s*</text>', r'###TERMO###\1 @\2@', texto)

|

# 5. Isolamento do Corpo do Dicionário
# Descarta o material introdutório segmentando o texto no primeiro neologismo válido ("abeta")
partes_inicio = re.split(r'###TERMO###abeta', texto, maxsplit=1)
if len(partes_inicio) > 1:
    texto = "###TERMO###abeta" + partes_inicio[1]
```

GLOSSÁRIO

NEOLOGISMOS

4

Processamento Iterativo de Neologismos

```
# 6. Segmentação e Processamento Iterativo dos Neologismos
blocos = re.split(r'###TERMO###', texto)

dicionario_final = {}
siglas_embutidas = {}

for bloco in blocos[1:]:
    bloco = bloco.strip()
    if len(bloco) < 3:
        continue

    # Separa o bloco nas três componentes primárias: [0] Termo, [1] C
    elems_gramatica = re.split(r'@(s\.m\.|s\.f\.)@', bloco, maxsplit=1)

    if len(elems_gramatica) > 2:
        termo = elems_gramatica[0].strip()
        genero = elems_gramatica[1].strip()
        resto = elems_gramatica[2].strip()

        # 6.1. Extração da Tradução para o idioma Inglês
        elems_ing = re.split(r'\[ing\];?s*', resto, maxsplit=1)
        trad_ing = ""
        if len(elems_ing) > 1:
            trad_ing = elems_ing[0].strip()
            resto = elems_ing[1].strip()

        # 6.2. Extração da Tradução para o idioma Espanhol
        elems_esp = re.split(r'\[esp\]s*|\[es\s', resto, maxsplit=1)
        trad_esp = ""
        if len(elems_esp) > 1:
            trad_esp = elems_esp[0].strip()
            resto = elems_esp[1].strip()
```

```
# 6.3. Extração da Definição (Isolando citações e exemplos práticos)
# O delimitador baseia-se na presença de aspas seguidas ou precedidas de pontuação
elems_citacao = re.split(r'\s*[""]\s*[\\...]+\| \s*[\\...]+\s*[""]', resto, maxsplit=1)
descricao_completa = elems_citacao[0].strip()
```

GLOSSÁRIO

NEOLOGISMOS

5

Extração de Siglas

```
# 6.4. Extração de Sigla
# Captura a sigla apenas se a definição iniciar pelo padrão "Sigla: XYZ"
sigla = ""
match_sigla = re.match(r'^Sigla:\s*([A-Z0-9\-\-])\s+(.*)', descricao_completa)
if match_sigla:
    sigla = match_sigla.group(1)
    descricao_completa = match_sigla.group(2)
```

GLOSSÁRIO

NEOLOGISMOS

6

Exportação dos JSONs

```
# 8. Exportação dos Dados Estruturados
with open("jsons_temporarios/neologismos_temp.json", "w", encoding="utf-8") as f_out:
    json.dump(dicionario_final, f_out, indent=4, ensure_ascii=False)

# Exportar as Siglas
siglas_ordenadas = dict(sorted(siglas_embutidas.items(), key=lambda item: item[0].lower()))
json_siglas_final = {
    "Siglas": siglas_ordenadas
}
with open("jsons_temporarios/siglas_embutidas_neologismos.json", "w", encoding="utf-8") as f_out_siglas:
    json.dump(json_siglas_final, f_out_siglas, indent=4, ensure_ascii=False)

print(f"Concluído! Foram extraídos {len(dicionario_final)} conceitos.")
print(f"Bônus: {len(siglas_ordenadas)} siglas embutidas foram extraídas para um ficheiro separado!")
return dicionario_final

extrair_neologismos("dados/glossario_neologismos_saude.xml")
```


GLOSSÁRIO

NEOLOGISMOS

7

neologismos.json

```
},
"espectroscopia de ressonância nuclear magnética": {
  "descricao": "Método utilizado para a análise da estrutura dimensional de uma proteína",
  "traducoes": {
    "EN": "spectroscopy of nuclear magnetic resonance",
    "ES": "espectroscopia de ressonância magnética nuclear"
  }
},
"esquistossomicida": {
  "descricao": "Medicamento utilizados no combate à esquistossomose, doença que afeta o ser humano",
  "traducoes": {
    "EN": "esquistossomicida",
    "ES": "esquistossomicida"
  }
},
"estímulo aversivo": {
  "descricao": "Designação para algo como um ambiente, um som, uma imagem ou uma luz que provoque medo",
  "traducoes": {
    "EN": "aversive stimulation",
    "ES": "estímulo aversivo"
  }
},
}
```

EXTRAÇÃO DE SIGLAS E ABREVIATURAS

EXTRAÇÃO PADRONIZADA DE SECÇÕES
INTRODUTÓRIAS

ou

RESGATE DE ENTIDADES EMBUTIDAS

AGREGAÇÃO E RESOLUÇÃO DE COLISÕES

```
526 > TrabalhoPrático1 > jsons_temporarios > {} abreviaturas_gerais.js
{
  "Abreviaturas": {
    "(en-US)": "English, United States",
    "(es-AR)": "Spanish, Argentina",
    "(es-CU)": "Spanish, Cuba",
    "(pt-BR)": "Portuguese, Brazil",
    "(pt-PT)": "Portuguese, Portugal",
    "(syn.)": "synonym",
    "a": "adjectivo",
    "abrev": "abreviatura",
    "AR": "Arabic",
    "arc.": "forma arcaica",
    "Br.": "português do Brasil",
    "CHEM": "Chemical & Materials Technology",
    "col.": "forma coloquial",
    "cult.": "forma culta",
    "DE": "German",
    "EE. UU.": "inglês americano",
    "ENVR": "Environmental & Safety Engineering",
    "ES": "Spanish",
    "et al.": "e outros",
    "f": "substantivo feminino",
    "FR": "French",
    "JA": "Japanese",
    "KO": "Korean",
    "lit.": "forma literaria",
    "loc": "locución",
    "m": "substantivo masculino",
    "MEDT": "Medical Science & Technology"
```

```
"Siglas": {
  "AB": "Atenção Básica",
  "ABEn": "Associação Brasileira de Enfermagem",
  "ADT": "Assistência Domiciliar Terapêutica",
  "AFE": "Autorização de Funcionamento de Empresa",
  "AIDPI": "Atenção Integrada às Doenças Prevalentes na Infância",
  "AIDS": "Síndrome da Imunodeficiência Adquirida",
  "AIH": "Autorização de Internação Hospitalar",
  "AIS": "Ações Integradas de Saúde",
  "ANCED": "Associação Nacional de Centros de Defesa",
  "ANS": "Agência Nacional de Saúde",
  "ANVISA": "Agência Nacional de Vigilância Sanitária",
  "APAC": "Autorização de Procedimentos de Alto Custo",
  "APH": "Assistência Pré-Hospitalar",
  "ASAJ": "Área de Saúde do Adolescente e do Jovem",
  "AVCI": "Acidente vascular cerebral isquêmico",
  "BD-SIA/SUS": "Banco de Dados Nacional do Sistema de Informações Amb",
  "BLH": "Banco de Leite Humano",
  "BPF": "Boas Práticas de Fabricação",
  "BPPH": "Banco de Preços Praticados na Área Hospitalar",
  "BPS": "Banco de Preços em Saúde",
  "BSE": "Encefalopatia espongiforme bovina",
  "BVS": "Biblioteca Virtual em Saúde",
  "CAF": "Cirurgia de alta Frequência",
  "CAPS": "Centro de Assistência Psicossocial",
  "CAT": "Comunicação de Acidente de Trabalho",
  "CBO": "Conselho Brasileiro de Oftalmologia",
  "CCIH": "Comissão de Controle de Infecção Hospitalar",
  "CCPDM": "Controle de Cadeia Produtiva e de Distribuição de Medicame",
  "CCPDS": "Controle de Cadeia Produtiva e de Distribuição de Substânc",
  "CDMS": "Comitê de Desburocratização do Ministério da Saúde",
  "CENADI": "Centro Nacional de Armazenagem e Distribuição de Imunobi",
  "CENEPI": "Centro Nacional de Epidemiologia",
  "CES": "Conselho Estadual de Saúde",
  "CFT": "Comissão de Farmácia e Terapêutica",
  "CIB": "Comissão Intergestores Bipartite",
  "CID": "Classificação Internacional de Doenças",
  "CIRH": "Comissão Intersetorial de Recursos Humanos",
  "CIST": "Comissão Intersetorial de Saúde do Trabalhador",
  "CIT": "Comissão Intergestores Tripartite",
  "CMC": "Sistema Central de Marcação de Consultas",
  "CNEP": "Comissão Nacional de Ética em Pesquisa"
```



JUNÇÃO DOS JSONS



NORMALIZAÇÃO ESTRUTURAL

Criação de um "Molde" para garantir que todos os conceitos dos 4 PDFs partilhavam as mesmas chaves (ex: preenchendo campos vazios com "Não identificado").

```
def normalizar_conceito(dados_originais):  
    """  
    Aplica o 'Molde' a um conceito. Se o conceito original não tiver uma chave,  
    ela é criada com o valor de defeito para garantir coerência estrutural (Schema Enforcement).  
    """  
    molde = {  
        "categoria": "Categoria não identificada",  
        "sinonimos": "Sinónimos não identificados",  
        "variantes": "Variantes não identificadas",  
        "descricao": "Descrição não identificada",  
        "traducoes": "Traduções não identificadas"  
    }  
  
    conceito_normalizado = {}  
    for chave, valor_defeito in molde.items():  
        # Se a chave existir no original e não for uma lista vazia ou string vazia, mantém.  
        # Caso contrário, aplica o valor por defeito.  
        if chave in dados_originais and dados_originais[chave] and dados_originais[chave] != []:  
            conceito_normalizado[chave] = dados_originais[chave]  
        else:  
            conceito_normalizado[chave] = valor_defeito  
  
    return conceito_normalizado
```


TRATAMENTO DE COLISÕES

- **Fusão por Match:** Se os metadados eram iguais, as descrições eram somadas.
- **Homonímia:** Se as origens eram diferentes, as chaves desdobravam-se (ex: (1) ampola, (2) ampola) para evitar perda de dados.

```
def adicionar_ao_dicionario(dicionario_destino, conceito, dados_novos):  
    """  
    Insere o conceito no dicionário final. Trata automaticamente repetições:  
    - Funde as descrições se todos os outros parâmetros forem idênticos.  
    - Cria homônimos (ex: (1) conceito, (2) conceito) se vierem de línguas/estruturas diferentes.  
    """  
    chaves_existentes = []  
  
    if conceito in dicionario_destino:  
        chaves_existentes.append(conceito)  
    for k in dicionario_destino.keys():  
        if re.match(rf'^\\(\\d+\\)\\s+(re.escape(conceito))$', k):  
            chaves_existentes.append(k)  
  
    if not chaves_existentes:  
        # Inserção limpa: primeira vez que vemos o conceito  
        dicionario_destino[conceito] = dados_novos  
    else:  
        # O conceito já existe. Vamos verificar se faz match com algum existente  
        chave_correspondente = None  
        for chave in chaves_existentes:  
            dados_existentes = dicionario_destino[chave]  
            if (dados_existentes["categoria"] == dados_novos["categoria"] and  
                dados_existentes["sinonimos"] == dados_novos["sinonimos"] and  
                dados_existentes["variantes"] == dados_novos["variantes"] and  
                dados_existentes["traducoes"] == dados_novos["traducoes"]):  
                chave_correspondente = chave  
                break  
  
        if chave_correspondente:  
            # Match perfeito: Fundir descrições  
            desc_atual = dicionario_destino[chave_correspondente]["descricao"]  
            nova_descricao = dados_novos["descricao"]  
  
            if nova_descricao != "Descrição não identificada" and nova_descricao not in desc_atual:  
                if desc_atual == "Descrição não identificada":  
                    dicionario_destino[chave_correspondente]["descricao"] = nova_descricao  
                elif not desc_atual.startswith("(1)":  
                    dicionario_destino[chave_correspondente]["descricao"] = f"(1) {desc_atual} (2) {nova_descricao}"  
                else:  
                    qtd_existentes = len(re.findall(r'\\(\\d+\\)', desc_atual))  
                    dicionario_destino[chave_correspondente]["descricao"] = f"{desc_atual} ({qtd_existentes + 1}) {nova_descricao}"  
            else:  
                # Homonímia (origens/línguas diferentes): separar em (1), (2), etc.  
                if conceito in chaves_existentes:  
                    dicionario_destino[f"(1) {conceito}"] = dicionario_destino.pop(conceito)  
                    chaves_existentes.remove(conceito)  
                    chaves_existentes.append(f"(1) {conceito}")  
  
                proximo_numero = len(chaves_existentes) + 1  
                nova_chave = f"({proximo_numero}) {conceito}"  
  
                dicionario_destino[nova_chave] = dados_novos
```


ESTRUTURA DO DICIONÁRIO FINAL

```
{
  "Glossário de Medicina e Saúde": {
    "Siglas": {
      "AB": "Atenção Básica",
      "ABEn": "Associação Brasileira de Enfermagem",
      "ADT": "Assistência Domiciliar Terapêutica",
      "AFE": "Autorização de Funcionamento de Empresa",
      "AIDPI": "Atenção Integrada às Doenças Prevalentes na Infância",
      "AIDS": "Síndrome da Imunodeficiência Adquirida",
      "AIH": "Autorização de Internação Hospitalar",
```

```
    },
    "Abreviaturas": {
      "(en-US)": "English, United States",
      "(es-AR)": "Spanish, Argentina",
      "(es-CU)": "Spanish, Cuba",
      "(pt-BR)": "Portuguese, Brazil",
      "(pt-PT)": "Portuguese, Portugal",
      "(syn.)": "synonym",
      "a": "adxectivo",
```

```
    },
    "Conceitos": {
      "á": {
        "categoria": [
          "Anatomía"
        ],
        "sinonimos": "Sinónimos não identificados",
        "variantes": "Variantes não identificadas",
        "descricao": "Descrição não identificada",
        "traducoes": {
          "ES": "ala",
          "EN": "wing",
          "PT": "asa",
          "LA": "ala"
        }
      }
    }
  },
}
```

- A Raiz: **"Glossário de Medicina e Saúde"**.
- O Preâmbulo: **"Siglas"** e **"Abreviaturas"** no topo como tabelas de consulta rápida.
- O Corpo: **"Conceitos"** estruturados e uniformizados.

CONCLUSÃO

1

Eficácia do PLN e Regex

Transformação bem-sucedida de quatro glossários despadronizados e ruidosos num único ficheiro JSON estruturado e pesquisável.

3

Otimização de Consulta

A criação de dicionários transversais de Siglas e Abreviaturas no preâmbulo do ficheiro enriqueceu e facilitou a navegação dos dados.

2

Arquitetura Modular Robusta

A utilização de scripts dedicados aliados a um *pipeline* de fusão garantiu uma extração limpa e a eliminação de conceitos duplicados.

4

Resiliência Técnica

O desenvolvimento de algoritmos de pós-processamento permitiu contornar as falhas nativas da conversão PDF e cumprir a consolidação terminológica.